Chapter 10 — Directed Graphs

Luca De Vecchis

Jan 2026

Contents

1 Directed Graphs **2**

2 Directed Paths **2**
2.1 Why Directed Paths Matter . . . 3

3 Directed Cycles **3**
3.1 Key Properties . . . 3
3.2 Degrees and Cycles . . . 3
3.3 Tournaments . . . 4
3.4 Importance of Directed Cycles . . . 4

4 A Job Sequencing Problem **5**
4.1 Graph Representation . . . 6
4.2 Applications and Importance . . . 6

5 Designing an Efficient Computer Drum **7**
5.1 Graph Representation . . . 7

6 Making a Road System One-Way **8**
6.1 Key Concepts . . . 8
6.2 Nash-Williams Generalization . . . 9

7 Ranking the Participants in a Tournament **10**
7.1 Graph Representation . . . 10
7.2 Ranking Methods . . . 10

1 Directed Graphs

Definition 1.1 (Directed Graph)**.** A **directed graph** (or **digraph**) is an ordered pair $G = (V, E)$, where:

- V is a finite set of *vertices* or *nodes*.
- $E \subseteq V \times V$ is a set of *directed edges* (or *arcs*), where each edge is an ordered pair (u, v) with $u, v \in V$. The edge (u, v) is considered to be directed from u to v.

Remark 1.1. Unlike undirected graphs, in a digraph the edge (u, v) is *not* the same as (v, u). This directionality allows modeling asymmetric relationships, such as precedence in tasks, one-way streets, or competitions.

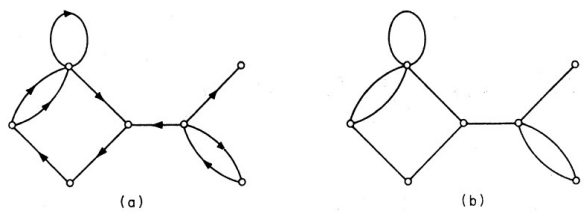

Figure 10.1. (*a*) A digraph D; (*b*) the underlying graph of D

This simple example illustrates how digraphs can naturally represent cyclic relationships. Directed graphs can be further classified based on the presence of cycles, connectivity, and other structural properties.

2 Directed Paths

Definition 2.1 (Directed Path)**.** A **directed path** in a digraph $D = (V, A)$ from vertex $v_1 \in V$ to vertex $v_k \in V$ is a sequence of vertices

$$(v_1, v_2, \ldots, v_k)$$

such that $(v_i, v_{i+1}) \in A$ for each $i = 1, 2, \ldots, k - 1$.

Definition 2.2 (Simple Path)**.** A directed path is called **simple** if all its vertices are distinct. If the first and last vertices coincide, it forms a cycle.

Remark 2.1. Directed paths are crucial for understanding reachability in digraphs. If there exists a directed path from u to v, then v is said to be *reachable* from u.

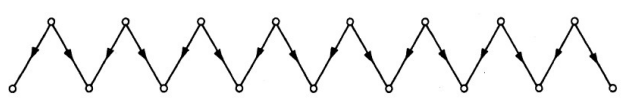

Figure 10.3

2.1 Why Directed Paths Matter

Directed paths provide the foundation for analyzing:

- Reachability in networks.
- Task scheduling and dependencies.
- Data and information flow.
- Connectivity analysis in road systems or tournaments.

3 Directed Cycles

A **directed cycle** in a digraph $D = (V, A)$ is a directed path

$$(v_1, v_2, \ldots, v_k, v_1)$$

with $k \geq 2$, where all vertices $v_1, \ldots, v_k$ are distinct, and the arc $(v_k, v_1) \in A$ closes the path into a cycle.

3.1 Key Properties

- The direction of arcs matters: traversing arcs in the opposite direction does not form a valid directed cycle.
- If a digraph contains no directed cycles, it is called a **Directed Acyclic Graph (DAG)**.
- DAGs are central to scheduling problems (§10.4) and other applications where dependencies must not form loops.
- The presence of a directed cycle indicates circular dependencies or feedback loops.

3.2 Degrees and Cycles

For each vertex v in a directed cycle:

$$d^+(v) \geq 1, \quad d^-(v) \geq 1$$

because every vertex has at least one incoming and one outgoing arc that participates in the cycle.

Example

Consider the digraph:

$$V = \{1, 2, 3\}, \quad A = \{(1, 2), (2, 3), (3, 1)\}.$$

$$\begin{tikzcd}
1 \arrow[r] & 2 \arrow[d] \\
& 3 \arrow[ul]
\end{tikzcd}$$

- The sequence $(1, 2, 3, 1)$ is a directed cycle.
- Each vertex has out-degree $d^+(v) = 1$ and in-degree $d^-(v) = 1$.
- This digraph is cyclic; every vertex participates in the cycle.

3.3 Tournaments

Definition 3.1 (Tournament)**.** A **tournament** is a digraph where, for every pair of distinct vertices $u \neq v$, exactly one of (u, v) or (v, u) is present. This models a round-robin competition where every pair of participants competes once, and there is a winner for each match.

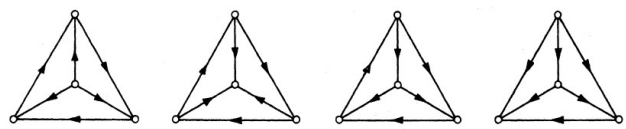

Figure 10.4. The tournaments on four vertices

Remark 3.1. In tournaments, directed cycles naturally appear and indicate circular wins:

- If (A, B), (B, C), and (C, A) are all arcs in the tournament, then A beats B, B beats C, and C beats A — a *cyclic ranking.*
- These cycles make ranking participants more complex because no participant is strictly dominant over all others.

3.4 Importance of Directed Cycles

- Detecting cycles is crucial in job scheduling (§10.4). A cycle indicates circular dependencies, which prevent any feasible schedule.
- Cycles in DAGs are forbidden; detecting them is a primary step before applying topological sorting.
- In tournaments (§10.7), cycles illustrate the presence of circular wins, which may require tie-breaking methods or advanced ranking algorithms.
- In network theory, cycles indicate feedback loops or potential infinite loops in routing or information flow.

Remark 3.2. Algorithms like depth-first search (DFS) can efficiently detect directed cycles. In tournaments, cycles of length 3 or more are common and provide insights into competition structure.

4 A Job Sequencing Problem

The job sequencing problem involves scheduling a set of jobs subject to precedence constraints. Each job must be completed after all its prerequisite jobs have been finished. Formally:

- Let $J = \{J_1, J_2, \ldots, J_n\}$ be a set of n jobs.
- Let the precedence constraints be represented as a set of directed arcs A, where $(J_i, J_j) \in A$ means *job J_i must be completed before job J_j*.
- The goal is to find a sequence of jobs $(J_{i_1}, J_{i_2}, \ldots, J_{i_n})$ that respects all constraints.

Step 1 Construct a digraph D with vertices $v_1, v_2, \ldots, v_n$, such that $(v_i, v_j) \in A$ if and only if $t_{ij} \leq t_{ji}$. By definition, D contains a spanning tournament.

Step 2 Find a directed Hamilton path $(v_{i_1}, v_{i_2}, \ldots, v_{i_n})$ of D (exercise 10.4.1), and sequence the jobs accordingly.

Since step 1 discards the larger half of the adjustment matrix $[t_{ij}]$, it is a reasonable supposition that this method, in general, produces a fairly good job sequence. Note, however, that when the adjustment matrix is symmetric, the method is of no help whatsoever.

As an example, suppose that there are six jobs J_1, J_2, J_3, J_4, J_5 and J_6 and that the adjustment matrix is

	J_1	J_2	J_3	J_4	J_5	J_6
J_1	0	5	3	4	2	1
J_2	1	0	1	2	3	2
J_3	2	5	0	1	2	3
J_4	1	4	4	0	1	2
J_5	1	3	4	5	0	5
J_6	4	4	2	3	1	0

The sequence $J_1 \to J_2 \to J_3 \to J_4 \to J_5 \to J_6$ requires 13 units in adjustment time. To find a better sequence, construct the digraph D as in step 1 (figure 10.8).

$(v_1, v_6, v_3, v_4, v_5, v_2)$ is a directed Hamilton path of D, and yields the sequence

$$J_1 \to J_6 \to J_3 \to J_4 \to J_5 \to J_2$$

which requires only eight units of adjustment time. Note that the reverse sequence

$$J_2 \to J_5 \to J_4 \to J_3 \to J_6 \to J_1$$

is far worse, requiring 19 units of adjustment time.

4.1 Graph Representation

- Represent each job as a vertex in a digraph $D = (V, A)$, with $V = J$.
- Draw a directed edge (J_i, J_j) if job J_i precedes J_j.
- The resulting digraph is a **directed acyclic graph** (DAG) if the precedence constraints are consistent (i.e., no circular dependencies exist).

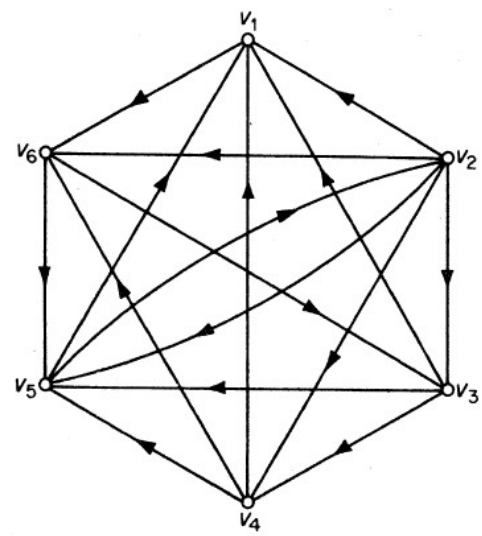

Figure 10.8

Topological Sorting

Definition 4.1 (Topological Sort)**.** A **topological sort** of a DAG is a linear ordering of its vertices such that for every edge (u, v), vertex u appears before vertex v in the ordering.

Remark 4.1. Topological sorting provides a valid sequence for executing jobs while respecting all precedence constraints. If a directed cycle exists, topological sorting is impossible, indicating that the constraints are inconsistent.

4.2 Applications and Importance

- Task scheduling in project management (e.g., PERT and CPM techniques).
- Instruction scheduling in computing to respect data dependencies.
- Production line optimization where certain steps must precede others.
- Any workflow that can be modeled as a DAG of dependencies.

Remark 4.2. The key insight is that a DAG allows a flexible ordering of tasks, but detecting cycles is essential to avoid impossible schedules. Algorithms for topological sorting, such as Kahn's algorithm or depth-first search (DFS)-based methods, provide efficient solutions.

5 Designing an Efficient Computer Drum

A computer drum is a type of early memory storage device where data is stored in a sequence of sectors along a rotating drum. Accessing data takes time proportional to the rotation distance from the current position.
The design problem is to **arrange data on the drum and schedule read/write operations efficiently** to minimize average access time. This can be modeled using a digraph:

- Each memory sector or data block is represented as a vertex.
- A directed arc (u, v) represents the operation of accessing sector v immediately after sector u.

5.1 Graph Representation

- Let $V = \{S_1, S_2, \ldots, S_n\}$ be the set of memory sectors.
- Let $A \subseteq V \times V$ be the set of directed arcs representing the desired access sequence.
- The digraph $D = (V, A)$ models all required accesses while capturing their order.

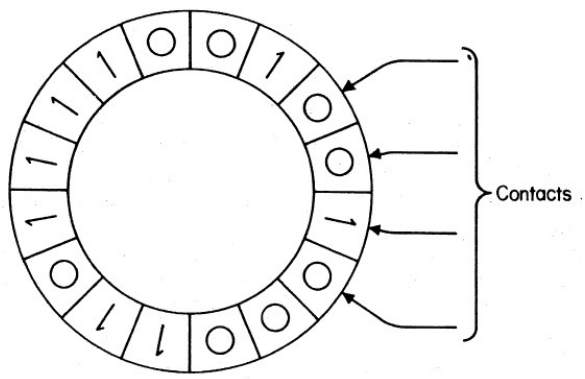

Figure 10.9. A computer drum

Optimization Goals

The main objective is to **minimize the total or average rotation distance** between consecutive memory accesses. Graph-theoretic considerations include:

- Finding a *directed path* that visits all required sectors efficiently (analogous to a Hamiltonian path).
- Avoiding repeated rotations over the same sectors unnecessarily.
- Using *cycles* to model repeated or recurring access patterns, ensuring that frequently accessed data are close to each other on the drum.

Graph-Theoretic Insights

- Frequent accesses between two sectors should ideally correspond to *short arcs* on the drum.
- The problem can be viewed as a *weighted digraph*, where each arc weight represents the rotation distance.
- Finding the minimal total access time is analogous to *finding an optimal directed path or cycle* that visits all vertices according to demand.

Remark 5.1. Although computer drums are largely historical, the underlying digraph model illustrates the broader principle: *graph-based representations can optimize sequential access problems with asymmetry in movement or cost.*

6 Making a Road System One-Way

The problem is: Given an undirected road network, can we assign a direction to each street to make all intersections reachable from every other intersection using the one-way streets?
Formally, given a connected undirected graph $G = (V, E)$ representing streets and intersections:

- Vertices V correspond to intersections.
- Edges E correspond to streets (initially undirected).

We want a *strongly connected digraph* $D = (V, A)$ by orienting each edge $e \in E$.

6.1 Key Concepts

Definition 6.1 (2-Edge-Connected Graph)**.** An undirected graph is **2-edge-connected** if it remains connected after removing any single edge.

Theorem 6.1 (Robbins, 1939)**.** An undirected connected graph can be oriented to form a strongly connected digraph if and only if it is 2-edge-connected.

Remark 6.1. This theorem gives a precise necessary and sufficient condition:

- If the graph has a bridge (an edge whose removal disconnects the graph), no orientation can make it strongly connected.
- If the graph is 2-edge-connected, there exists at least one orientation that produces a strongly connected digraph.

Construction of Strong Orientation

- Start with any 2-edge-connected undirected graph G.
- Use a depth-first search (DFS) or ear decomposition to orient edges sequentially.
- Every edge is assigned a direction to ensure that from any vertex u there is a directed path to any other vertex v.

6.2 Nash-Williams Generalization

Theorem 6.2 (Nash-Williams, 1960)**.** Let G be a connected graph. For any integer $k \geq 1$, G can be oriented to contain k arc-disjoint strongly connected spanning subdigraphs if and only if G is $2k$-edge-connected.

Nash-Williams (1960) has generalised Robbins' theorem by showing that every $2k$-edge-connected graph G has a k-arc-connected orientation. Although the proof of this theorem is difficult, the special case when G has an Euler trail admits of a simple proof.

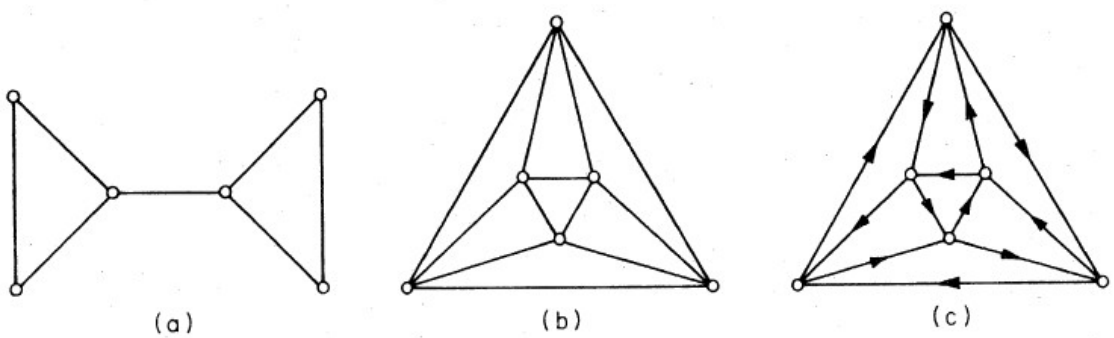

Figure 10.12. (*a*) G_1; (*b*) G_2; (*c*) D_2

Figure 1: Enter Caption

Remark 6.2. For $k = 1$, this reduces to Robbins' theorem. This generalization is useful in networks requiring multiple redundant paths (e.g., fault-tolerant one-way systems).

Remark 6.3. Robbins' theorem formalizes the intuition that bridges "block" strong orientations, and 2-edge connectivity is both necessary and sufficient. Nash-Williams extends this for multiple arc-disjoint strong subdigraphs, useful in highly reliable traffic or communication networks.

7 Ranking the Participants in a Tournament

Consider a round-robin tournament in which each participant plays against every other participant exactly once. Given the outcomes of the games, the task is to rank the participants in a fair and consistent manner.

7.1 Graph Representation

- Each participant is represented as a vertex in a digraph $D = (V, A)$.
- An arc $(P_i, P_j) \in A$ indicates that participant P_i defeats participant P_j.
- By definition, a tournament is a complete asymmetric digraph: for every pair of distinct vertices $u \neq v$, exactly one of (u, v) or (v, u) is present.

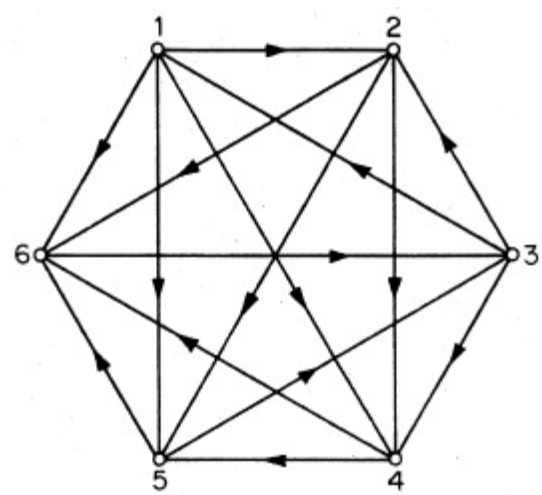

Figure 10.13

7.2 Ranking Methods

Hamiltonian Path Approach One simple method is to find a **directed Hamiltonian path** in the tournament and rank participants according to their positions along the path. For example, the path $(3, 1, 2, 4, 5, 6)$ declares player 3 the winner, player 1 runner-up, and so on. However, most tournaments have many Hamiltonian paths (e.g., $(1, 2, 4, 5, 6, 3)$, $(1, 4, 6, 3, 2, 5)$), making this method ambiguous and insufficient for robust ranking.

Score Vector Method A more systematic approach uses **score vectors**:

- Compute the *first-level score vector* $\mathbf{s}_1$, where each entry counts the number of wins per player. In the example of six participants:

$$\mathbf{s}_1 = (4, 3, 3, 2, 2, 1)$$

- First-level scores may not distinguish players who defeated stronger opponents. For instance, players 2 and 3 have the same number of wins, but player 3 defeated higher-scoring opponents.

Second-Level and Iterated Scores .
To refine rankings, define *second-level scores*:

$$s_2(P_i) = \sum_{P_j \text{ defeated by } P_i} s_1(P_j)$$

In the example:

$$\mathbf{s}_2 = (8, 5, 9, 3, 4, 3)$$

Iterating this procedure produces higher-level score vectors $\mathbf{s}_3, \mathbf{s}_4, \ldots$, converging to a stable ranking:

$$\begin{aligned}
\mathbf{s}_3 &= (15, 10, 16, 7, 12, 9) \\
\mathbf{s}_4 &= (38, 28, 32, 21, 25, 16) \\
\mathbf{s}_5 &= (90, 62, 87, 41, 48, 32) \\
\mathbf{s}_6 &= (183, 121, 193, 80, 119, 87)
\end{aligned}$$

Adjacency Matrix and Perron-Frobenius Method Let A be the adjacency matrix of the tournament, where $A_{ij} = 1$ if P_i defeats P_j, and 0 otherwise. If the tournament is **diconnected** and has at least four vertices:

- The adjacency matrix A is *primitive*, meaning some power A^k has all positive entries.
- By the Perron-Frobenius theorem, A has a unique largest positive eigenvalue r, and the corresponding positive eigenvector $\mathbf{s}$ represents the relative strengths of the players.
- Normalizing $\mathbf{s}$ yields a vector of relative rankings. In the six-player example:

  $$r \approx 2.232, \quad \mathbf{s} \approx (0.238, 0.164, 0.231, 0.113, 0.150, 0.104)$$

  Ranking by $\mathbf{s}$ gives the order: $1, 3, 2, 5, 4, 6$.

Ranking Procedure for General Tournaments

1. In each diconnected component with four or more vertices, compute the Perron-Frobenius eigenvector to rank participants; for components with three vertices, rank all equally.
2. Linearly order the diconnected components according to dominance: if D_i dominates D_j, all arcs from D_i to D_j point toward D_j.

Remark 7.1. This procedure, due to Wei (1952) and Kendall (1955), guarantees a consistent ranking that accounts for both direct wins and the strength of defeated opponents, even in tournaments with cycles and complex win patterns.